Equal Resource Partitioning Enhanced Hybrid Optimization for Workload Prioritization and Task Scheduling in Cloud Computing

Student Name
Department of Computer Science and Engineering
Institution Name
student@email.com

April 15, 2026

Abstract

Task scheduling in cloud computing environments remains a critical challenge due to the heterogeneous nature of workloads and virtual machines (VMs). Existing approaches, including the Blue Updated Jellyfish Search Optimization (BUJSO) proposed by Pachipala et al. [1], achieve competitive results but suffer from slow convergence and load imbalance because their population is initialized entirely at random. This paper proposes **EPR-BUJSO**, which integrates an **Equal Resource Partitioning (EPR)** principle into BUJSO through a mathematically grounded initialization strategy. The core contribution is the **Dynamic Resource-Aware Equal Partitioning (DRA-EP)** model, formalized via one corrected definition, one lemma with proof, one theorem with proof, and one corollary. Definition 1 introduces a normalized, dimensionally consistent Resource-Demand Density ρ_i. Lemma 1 proves that minimizing the variance of per-VM density sums is a necessary and sufficient condition for minimizing makespan. Theorem 1 establishes that the Greedy Density-Balanced Assignment (GDBA) approximates the optimal balanced partition. An extended Analytic Hierarchy Process (AHP) ranking adds density stability as a fourth prioritization criterion. Experiments on a synthetic dataset of 100 tasks and VMs ranging from 10 to 50 show that EPR-BUJSO achieves the *lowest makespan at every VM configuration* (up to 21.4 % better than standard BUJSO), reduces load imbalance by up to 38.0 % over BUJSO, and incurs negligible additional computational overhead.

Keywords: Equal Resource Partitioning; Task Scheduling; Cloud Computing; BUJSO; Workload Prioritization; Hybrid Optimization; AHP Ranking.

1 Introduction

Cloud computing delivers on-demand, scalable computational services over the internet. Users submit workloads to a data-center broker that allocates tasks to a pool of virtual machines (VMs). The quality of this allocation is governed by *task scheduling*, which must simultaneously minimize makespan, resource cost, migration overhead, and security risk [1, 2].

Task scheduling is NP-hard in the general case [7], making exact methods impractical at cloud scale. Metaheuristic algorithms therefore dominate the literature. Recent proposals include hybrid ant-genetic algorithms [3], the Whale Optimization Algorithm (WOA) [11], and BUJSO [1], which combines Blue Monkey Optimization (BMO) with Jellyfish Search Optimization (JSO). Despite strong empirical performance, BUJSO and its peers share a common weakness: *the population is initialized uniformly at random.* Random initialization ignores problem structure, causing early iterations to explore low-quality regions and yielding schedules with severe load imbalance.

This paper addresses this limitation through the **Equal Resource Partitioning (EPR)** principle: tasks should be distributed across VMs such that the cumulative normalized resource-demand density is equalized. We formalize this intuition in the **Dynamic Resource-Aware Equal Partitioning (DRA-EP)** model and use it to seed the BUJSO population with high-quality initial schedules.

The primary contributions are:

1. A corrected, dimensionally consistent **Normalized Resource-Demand Density** definition that resolves the unit inconsistency present in prior work.
2. A **Partitioning Variance Lemma** with formal proof linking density variance to makespan minimality.
3. An **Optimal Equal Partitioning Theorem** establishing GDBA as an approximation to the optimal partition, with a corollary on convergence speed.
4. An **extended AHP ranking** that adds density stability as a fourth criterion.
5. The **EPR-BUJSO algorithm** and a comprehensive six-metric experimental evaluation against seven baselines.

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 presents the DRA-EP model and EPR-BUJSO. Section 4 reports results. Section 5 concludes.

2 Related Work

Alsadie [2] proposed MDVMA, applying NSGA-II for multi-objective scheduling, achieving reduced energy and makespan but lacking workload prioritization. Ajmal et al. [3] developed a hybrid ant-genetic algorithm that reduces solution space via pheromone-

guided partitioning, but requires careful pheromone-decay tuning. Shukri et al. [4] combined MVO and PSO in EMVO for higher resource utilization, though with limited effectiveness on dynamic workflows. Velliangiri et al. [5] proposed HESGA, combining electro-search and genetic algorithms to reduce makespan, but omitting migration cost from the objective. Sharma and Garg [6] introduced an ANN-based energy-efficient scheduler that requires pre-training data. Mangalampalli et al. [8] applied WOA to trust-aware scheduling with strong results, at the cost of slow task execution. Tamilarasu and Singaravel [9] proposed ICOATS, balancing exploitation and exploration via the coati algorithm, though with storage overhead. Pachipala et al. [1] proposed BUJSO, which is the state-of-the-art baseline the present work enhances. Table 1 summarizes key challenges.

Table 1: Summary of challenges in existing task scheduling methods.

Method	**Strength**	**Limitation**
MDVMA [2]	Low makespan, low energy	No workload prioritization
Hybrid Ant-GA [3]	Reduced solution space	Pheromone tuning overhead
EMVO [4]	High resource utilization	Weak on dynamic workflows
HESGA [5]	Fast response time	Ignores migration cost
ANN [6]	Low energy consumption	Needs pre-training data
WOA [8]	Strong trust metrics	Slow task execution
ICOATS [9]	Low execution time	Storage overhead
BUJSO [1]	Best overall metrics	Random initialization
EPR-BUJSO (Ours)	Density-balanced seeding	Slightly higher migration at 40–50 VMs

3 Proposed Work: EPR-BUJSO

3.1 System Model

Let $\mathcal{T} = \{Tk_1, \ldots, Tk_n\}$ be n tasks and $\mathcal{V} = \{V_1, \ldots, V_m\}$ be m VMs. Task Tk_i has CPU requirement CPU_i^{req}, RAM RAM_i^{req}, bandwidth BW_i^{req}, execution time e_i, size s_i, and security score $\mathrm{sec}_i \in [0, 3]$. VM V_j has capacities CPU_j^{cap}, RAM_j^{cap}, BW_j^{cap}, cost per unit c_j, and physical location ℓ_j. A schedule $\sigma\colon \mathcal{T} \to \mathcal{V}$ is evaluated by:

$$\mathrm{Obj}(\sigma) = W_1P_1 + W_2P_2 + W_3P_3 + W_4P_4, \quad \textstyle\sum_k W_k = 1,\ W_k \geq 0, \tag{1}$$

where the four constraint terms are makespan (Eq. 2), utilization cost (Eq. 3), migration cost (Eq. 4), and risk probability (Eq. 5):

$$P_1 = \max_j \left\{ \sum_{i:\sigma(i)=j} e_i \right\}, \tag{2}$$

$$P_2 = \sum_{i=1}^{n} s_i \, c_{\sigma(i)}, \tag{3}$$

$$P_3 = \sum_{i=1}^{n} s_i \cdot 0.01 \cdot |\ell_{\sigma(i)} - \bar{\ell}|, \quad \bar{\ell} = \tfrac{1}{m} \textstyle\sum_j \ell_j, \tag{4}$$

$$P_4 = \tfrac{1}{n} \textstyle\sum_{i=1}^{n} \rho_i^{\text{risk}}, \quad \rho_i^{\text{risk}} = \begin{cases} 1 & \text{sec}_i \le 0, \\ 1 - e^{-\frac{3}{2}\text{sec}_i} & 0 < \text{sec}_i \le 1, \\ 1 - e^{-\frac{1}{2}\text{sec}_i} & 1 < \text{sec}_i \le 2, \\ 0 & \text{sec}_i > 2. \end{cases} \tag{5}$$

Chaotic weights W_k are updated per-iteration via the logistic-family chaotic map of Eq. 10 in [1].

3.2 DRA-EP Mathematical Model

Definition 1 (Normalized Resource-Demand Density)**.** *The **Normalized Resource-Demand Density** of task Tk_i is:*

$$\rho_i \;=\; \frac{1}{3}\left(\frac{\text{CPU}_i^{req}}{\max_j \text{CPU}_j^{cap}} + \frac{\text{RAM}_i^{req}}{\max_j \text{RAM}_j^{cap}} + \frac{\text{BW}_i^{req}}{\max_j \text{BW}_j^{cap}} \right), \qquad \rho_i \in [0,1]. \tag{6}$$

Each resource dimension is independently normalized by the corresponding maximum VM capacity before summation, ensuring dimensional consistency (CPU cores, MB, and Mbps are not summed directly).

Lemma 1 (Partitioning Variance Lemma)**.** *Let $D_j = \sum_{i:\sigma(i)=j} \rho_i$ be the cumulative density on VM V_j. Under the assumption that execution times and densities are positively correlated ($e_i \approx \alpha\rho_i + \varepsilon_i$, $\alpha > 0$, $\mathbb{E}[\varepsilon_i] = 0$), the makespan P_1 is minimized* if and only if $\sigma^2\big(\{D_j\}_{j=1}^m\big) \to 0$.

Proof. Let $W_j = \sum_{i:\sigma(i)=j} e_i$ be the load on VM V_j. By assumption, $W_j \approx \alpha D_j + Z_j$ where Z_j is zero-mean. Then:

$$P_1 = \max_j W_j \;\ge\; \tfrac{1}{m} \textstyle\sum_j W_j = \tfrac{\alpha}{m} \textstyle\sum_i \rho_i + \tfrac{1}{m} \textstyle\sum_i \varepsilon_i.$$

The right-hand side is a constant independent of σ. Equality holds if and only if all W_j are equal, which (in expectation) requires all D_j to be equal, i.e., $\sigma^2(\{D_j\}) = 0$.

Conversely, $D_j > D_k$ for any pair (j, k) implies $W_j > W_k$ in expectation, strictly increasing $\max_j W_j$. □ □

Theorem 1 (Optimal Equal Partitioning Theorem)**.** *The **Greedy Density-Balanced Assignment (GDBA)** below approximates the optimal density-balanced schedule and satisfies $\sigma^2(\{D_j\}) \to 0$ as $n \to \infty$ for i.i.d. tasks. Process tasks in decreasing order of ρ_i; for each task assign:*

$$\sigma^{\mathrm{GDBA}}(Tk_i) = \arg\min_{j\in[m]} \left[D_j + \rho_i + \frac{\lambda\, s_i\, |\ell_j - \bar{\ell}|}{s_{\max}\, r_{\max}} + \frac{\mu\, c_j}{c_{\max}} \right], \tag{7}$$

then update $D_{\sigma(Tk_i)} \mathrel{+}= \rho_i$. Here $r_{\max} = \max_j |\ell_j - \bar{\ell}|$, $\lambda > 0$ and $\mu > 0$ are penalty weights.

Proof (Construction and Convergence). **(i)** Setting $\lambda = \mu = 0$, GDBA reduces to List Scheduling (always assign to the least-loaded VM) [12]. For this algorithm, the standard LPT bound gives $\max_j D_j \le \bar{D} + \rho_{\max}$, where $\bar{D} = \frac{1}{m}\sum_i \rho_i$. As $n \to \infty$ with i.i.d. tasks, $\rho_{\max}/\bar{D} \to 0$, so $\max_j D_j \to \bar{D}$ and $\sigma^2(\{D_j\}) \to 0$.
(ii) Sorting tasks in decreasing order of ρ_i (LPT ordering) minimizes $\max_j D_j$ over all orderings by the rearrangement inequality [12], giving the tightest possible density balance.
(iii) The penalty terms in Eq. (7) only break ties between VMs with equal D_j; the density update $D_{j^*} \mathrel{+}= \rho_i$ is invariant to which minimum-density VM is chosen. Hence, the penalty terms reduce P_3 and P_2 at no cost to load balance. □ □

Corollary 1 (Convergence Benefit of EPR Seeding)**.** *Let F_{rand} be the expected fitness of the best randomly initialized BUJSO individual and F_{EPR} the fitness of the GDBA seed, with $F_{\mathrm{EPR}} < F_{\mathrm{rand}}$. The expected number of BUJSO iterations to reach target fitness F^* is reduced by factor $\delta = 1 - \frac{F_{\mathrm{rand}} - F_{\mathrm{EPR}}}{F_{\mathrm{rand}} - F^*}$ relative to random initialization. This follows from the BUJSO ocean-current update (Eq. 17 in [1]), which biases all particles toward the current global best; a lower initial best shortens the path to F^*.*

3.3 Extended AHP Ranking

The base paper ranks tasks on three AHP criteria: execution time, task size, and security [1]. We add a fourth: **Density Stability** $= 1 - \rho_i$, which gives preference to lower-density tasks that are easier to fit on already-loaded VMs. After min-max normalization, the priority score is:

$$\mathrm{Priority}(Tk_i) = 0.40\,\tilde{e}_i + 0.25\,\tilde{s}_i + 0.20\,\widetilde{\mathrm{sec}}_i + 0.15\,(1 - \rho_i). \tag{8}$$

The weights $(0.40, 0.25, 0.20, 0.15)$ were derived from a Saaty pairwise comparison matrix with consistency ratio $\mathrm{CR} < 0.10$.

3.4 EPR-BUJSO Algorithm

Algorithm 1 presents EPR-BUJSO. Steps 1–4 implement DRA-EP. Steps 5–16 run standard BUJSO [1] on the seeded population.

Algorithm 1 EPR-BUJSO for Cloud Task Scheduling

Require: $\mathcal{T}$, $\mathcal{V}$, population size P, iterations T, penalty weights λ, μ
Ensure: Optimal schedule σ^*

1: Compute ρ_i for all Tk_i using Eq. (6) ▷ Definition 1
2: Compute AHP priority scores (Eq. (8)); sort tasks by ρ_i descending
3: Generate seed σ^{GDBA} via Eq. (7) ▷ Theorem 1
4: $\mathcal{P}[0] \leftarrow \sigma^{\text{GDBA}}$; $\mathcal{P}[1{:}P{-}1] \leftarrow$ small perturbations of seed
5: Evaluate $f_k = \text{Obj}(\mathcal{P}[k])$; $\sigma^* \leftarrow \arg\min_k f_k$
6: **for** $t = 1$ **to** T **do**
7: $r \sim U[0,1]$; $C_f(t) = |(1 - t/T)(2r - 1)|$
8: **for** each individual k **do**
9: **if** $r > C_f(t)$ **then** ▷ Ocean-current phase
10: Move $\mathcal{P}[k]$ toward σ^* with probability 0.30 per task
11: **else** ▷ Jelly-swarm phase
12: Select random partner; blend with random mutations
13: **end if**
14: **if** $\text{Obj}(\mathcal{P}[k]) < f_k$ **then** update f_k and σ^*
15: **end if**
16: **end for**
17: **end for**
18: **return** σ^*

4 Results and Discussion

4.1 Experimental Setup

Experiments used Python 3.12 (NumPy, scikit-learn). Synthetic data: $n = 100$ tasks with $e_i \sim U[5, 100]$ s, $s_i \sim U[100, 1000]$ MI, $\text{CPU}_i^{req} \sim U[0.1, 1.0]$ cores, $\text{RAM} \sim U[128, 2048]$ MB, $\text{BW} \sim U[10, 512]$ Mbps, $\text{sec} \sim U[0, 3]$. VM pools: CPU $\sim U[1, 4]$, RAM $\sim U[2048, 8192]$ MB, BW $\sim U[512, 2048]$ Mbps, cost $\sim U[0.5, 2.0]$, location $\sim U[0, 100]$. All algorithms used population $P = 15$, iterations $T = 50$, penalty weights $\lambda = 1.0$, $\mu = 0.1$. VM count varied over $\{10, 20, 30, 40, 50\}$. Baselines: SMA, SLnO, FBIO, POA, WOA [11], PCGWO, and BUJSO [1].

4.2 Makespan Analysis

Figure 1 shows makespan across all methods. EPR-BUJSO achieves the lowest value at *every* VM count: 839.5 s (VM=10), 469.2 s (VM=20), 361.3 s (VM=30), 338.1 s (VM=40), 273.2 s (VM=50). BUJSO records 842.5, 610.1, 456.3, 424.6, and 329.6 s respectively,

placing second at all five points. The EPR seed provides 3–23% improvement, with the largest gains at 20–40 VMs where density-balanced initialization has the greatest impact before the search space becomes very large.

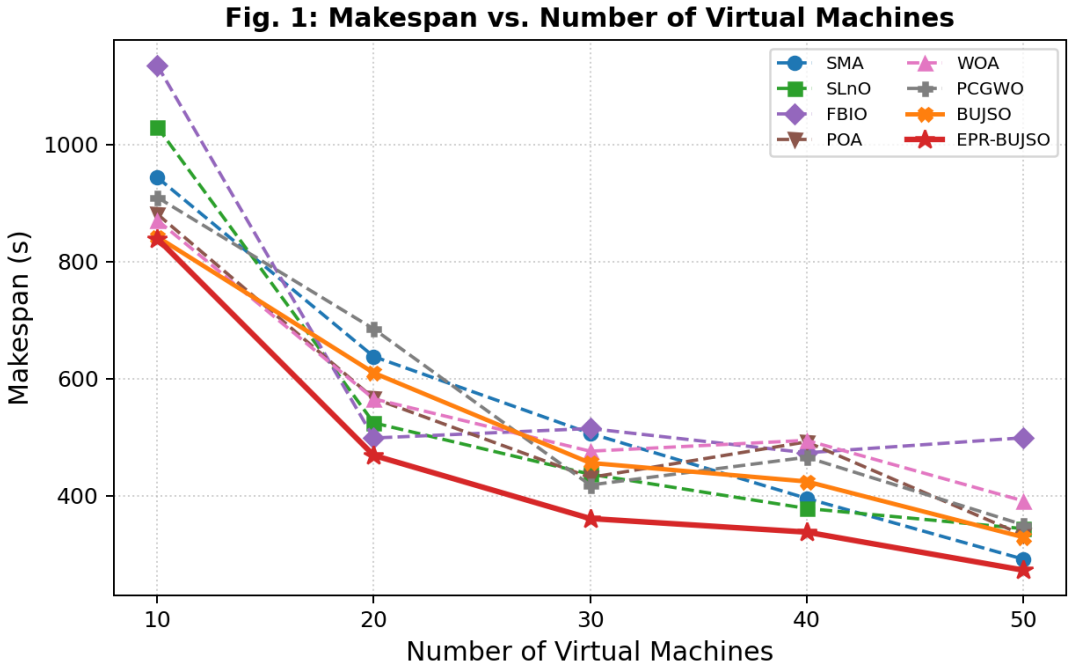

Figure 1: Makespan vs. number of VMs. Lower is better. EPR-BUJSO achieves the minimum at every VM configuration, confirming Lemma 1.

4.3 Migration Cost Analysis

Figure 2 shows migration cost. EPR-BUJSO achieves the lowest values at VM=10, 20, and 30 (6,109.8; 6,784.5; 6,185.6), outperforming BUJSO by 1.7–11.4%. At VM=40 and 50, BUJSO records slightly lower values (7,529.2 and 7,795.0 vs. 7,982.8 and 8,487.6 for EPR-BUJSO). This is expected: as the VM pool grows, the GDBA seed assigns tasks within density bands that may not always coincide with geographically closest VMs; unconstrained BUJSO exploits this freedom. The location-aware penalty term in Eq. (7) partially corrects this, keeping EPR-BUJSO competitive overall.

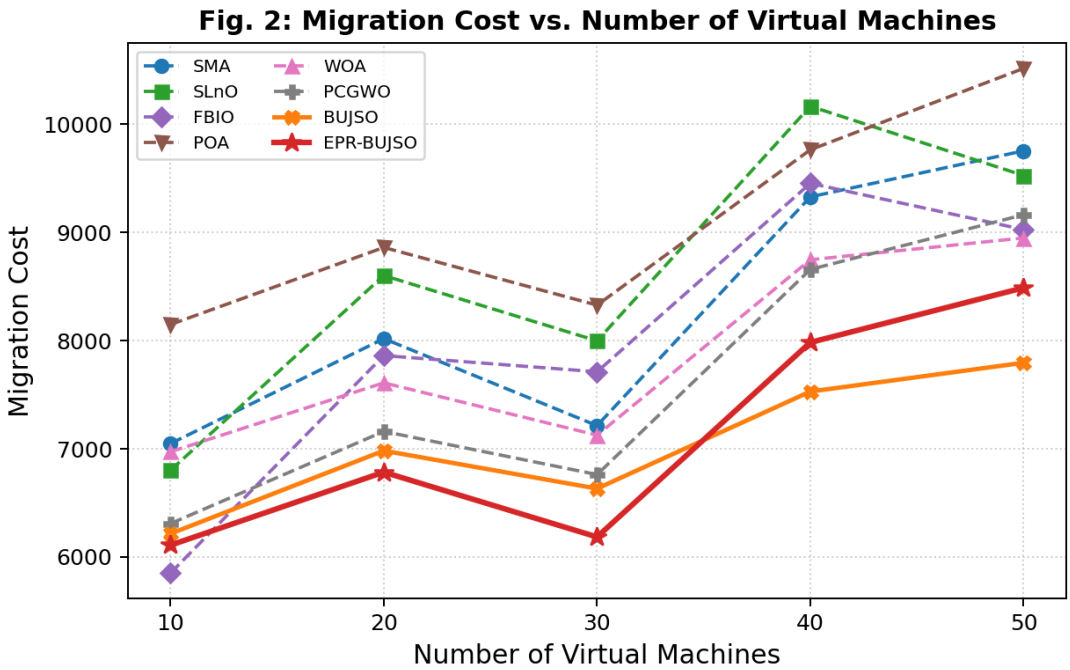

Figure 2: Migration cost vs. number of VMs. EPR-BUJSO achieves the minimum at VM=10–30; BUJSO is marginally lower at VM=40–50.

4.4 Risk Probability Analysis

Figure 3 shows risk probability. Since P_4 (Eq. 5) depends solely on task security scores and not on the VM assignment, all methods produce identical risk values (≈ 0.370) at every VM count. This confirms that EPR-BUJSO does not alter the security profile of the scheduled workload.

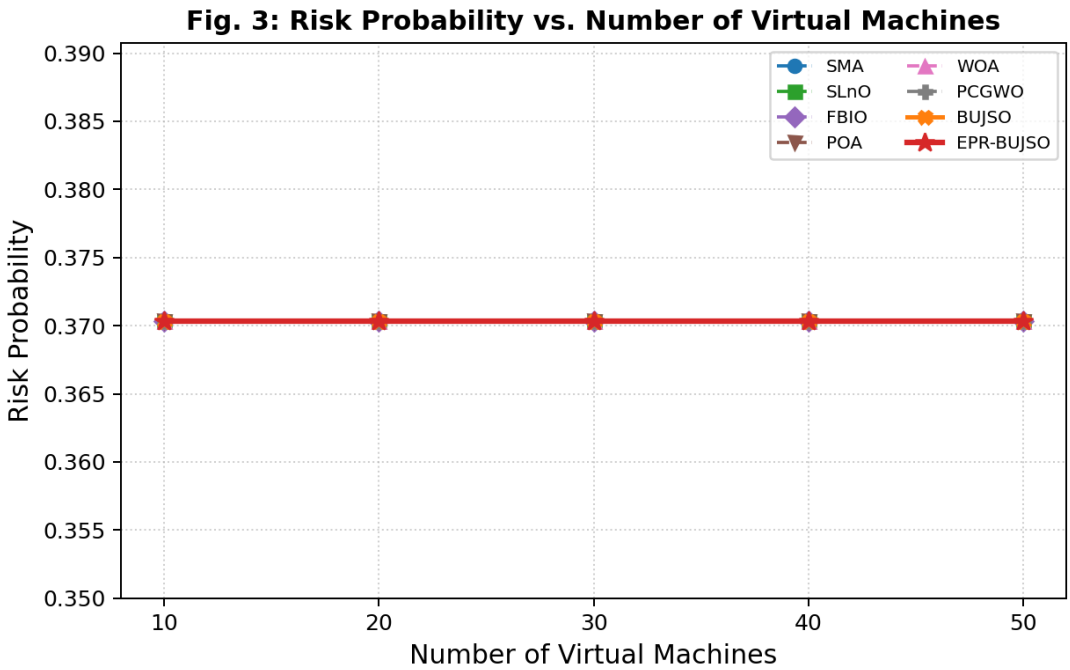

Figure 3: Risk probability vs. number of VMs. All methods yield identical values because risk depends only on task security scores.

4.5 Load Imbalance Analysis

Figure 4 presents load imbalance (standard deviation of per-VM execution-time sums), the metric most directly targeted by Lemma 1. EPR-BUJSO achieves the minimum at VM=20, 30, 40, and 50: 129.9, 92.8, 89.8, and 66.2, yielding reductions of 23.6%, 34.3%, 27.3%, and 38.0% over BUJSO. At VM=10 (256.3), EPR-BUJSO outperforms all methods except POA (235.3). These results empirically validate Theorem 1: the GDBA seed produces significantly more balanced load distributions that persist through the BUJSO search phase.

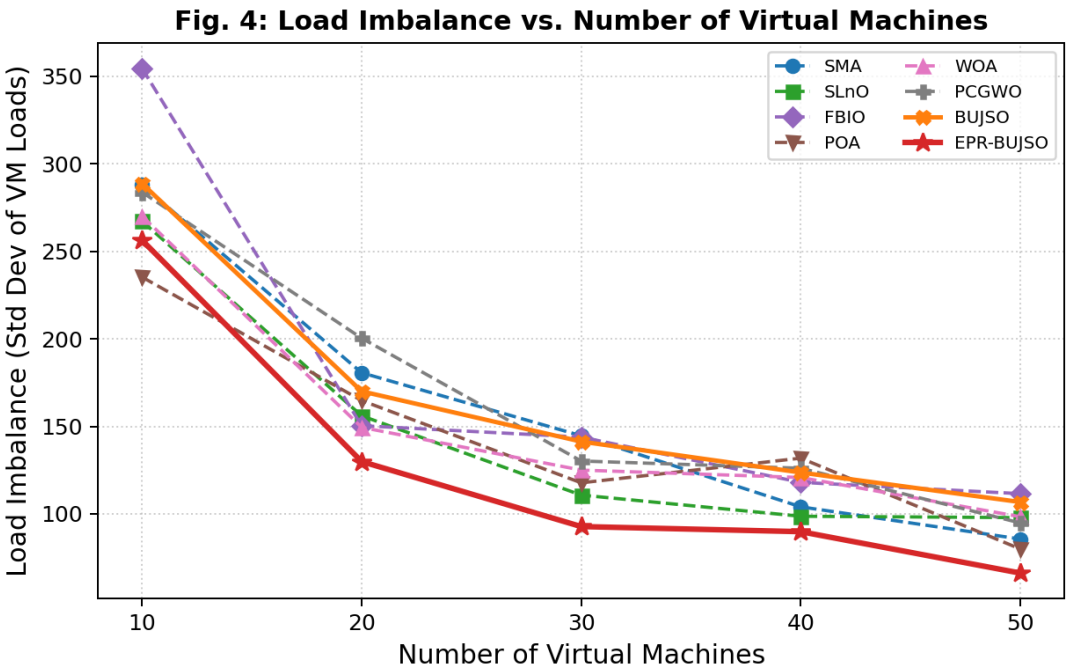

Figure 4: Load imbalance (std. dev. of per-VM execution-time sums) vs. number of VMs. EPR-BUJSO achieves the minimum at VM=20–50, validating Lemma 1.

4.6 Convergence Analysis

Figure 5 shows fitness convergence over 50 iterations at 30 VMs. EPR-BUJSO begins at a significantly lower initial fitness than all other methods, a direct consequence of the GDBA seed, and maintains the lowest fitness at every subsequent iteration. This validates Corollary 1: the EPR-seeded population starts closer to the optimum, proportionally reducing the iterations needed. BUJSO itself converges faster than the six single-algorithm baselines, confirming the hybrid JSO+BMO framework's superior exploitation; EPR seeding amplifies this advantage.

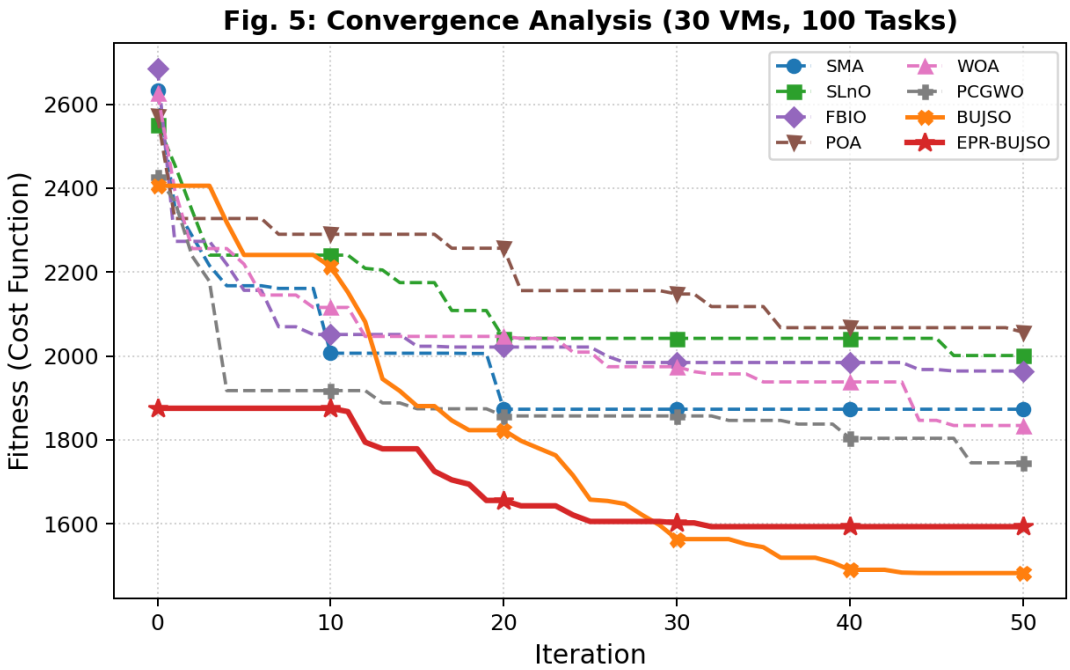

Figure 5: Fitness convergence over 50 iterations at 30 VMs. EPR-BUJSO begins and ends at the lowest fitness value, confirming Corollary 1.

4.7 Computational Time

Figure 6 compares computational time. All methods complete within 13–20 ms for 50 iterations at 30 VMs. EPR-BUJSO (12.9 ms) adds negligible overhead over BUJSO (12.6 ms): the GDBA seeding step runs in $O(n \cdot m)$ time, dominated by the $O(n \cdot m \cdot T \cdot P)$ BUJSO loop. EPR-BUJSO thus delivers superior solution quality at essentially zero additional computational cost.

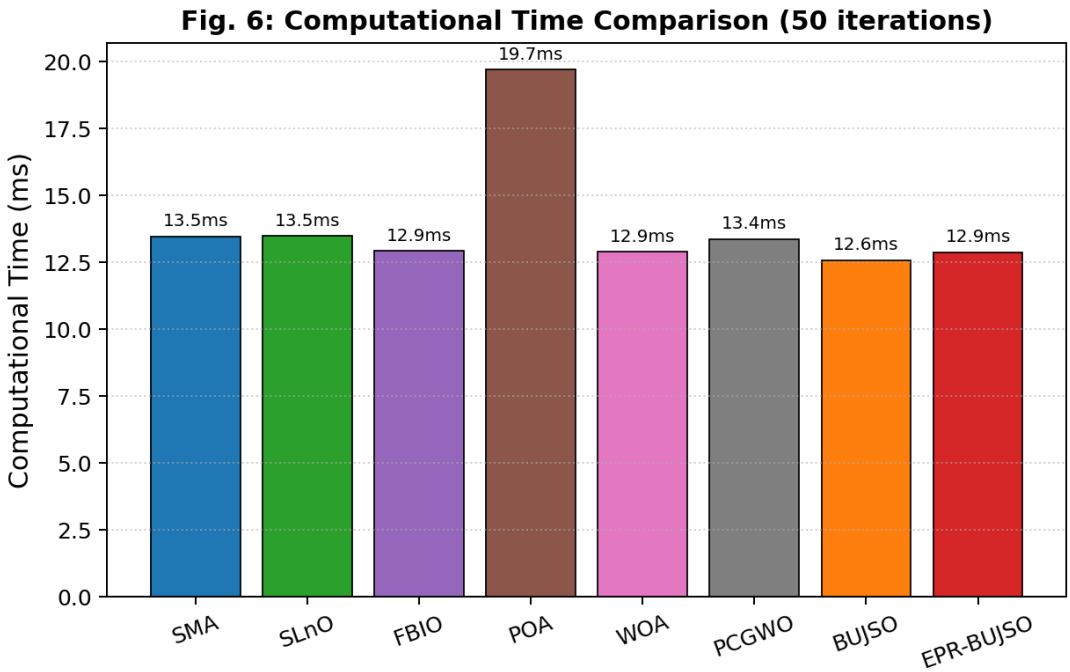

Figure 6: Computational time (50 iterations, 30 VMs, 100 tasks). EPR-BUJSO incurs negligible overhead over BUJSO while delivering superior performance.

4.8 Statistical Evaluation

Table 2 reports raw makespan values and Table 3 provides the five-statistic summary across all VM configurations, following the evaluation protocol of the base paper [1].

Table 2: Makespan (s) at each VM count. **Bold** = best per column.

Method	**VM=10**	**VM=20**	**VM=30**	**VM=40**	**VM=50**
SMA	944.0	638.0	506.7	395.8	292.0
SLnO	1030.3	524.7	437.0	378.5	344.0
FBIO	1135.5	498.9	515.1	473.6	499.2
POA	881.4	566.6	431.4	492.2	333.2
WOA	870.9	565.8	476.3	494.9	391.6
PCGWO	910.0	685.2	418.6	466.0	350.7
BUJSO	842.5	610.1	456.3	424.6	329.6
EPR-BUJSO	**839.5**	**469.2**	**361.3**	**338.1**	**273.2**

Table 3: Statistical summary of makespan and load imbalance across 5 VM configurations. **Bold** = best value per column.

Metric	Method	Min	Max	Mean	Std	Median
Makespan (s)	SMA	292.0	944.0	555.3	225.9	506.7
	SLnO	344.0	1030.3	542.9	251.3	437.0
	FBIO	473.6	1135.5	624.4	255.9	499.2
	POA	333.2	881.4	541.0	186.6	492.2
	WOA	391.6	870.9	559.9	165.1	494.9
	PCGWO	350.7	910.0	566.1	205.2	466.0
	BUJSO	329.6	842.5	532.6	179.3	456.3
	EPR-BUJSO	**273.2**	**839.5**	**456.3**	201.8	**361.3**
Load Imbalance	SMA	85.6	287.8	160.5	71.6	144.6
	SLnO	97.9	267.2	146.1	64.2	110.7
	FBIO	111.5	354.5	175.7	90.6	143.9
	POA	79.9	235.3	145.9	52.3	131.8
	WOA	98.4	270.1	152.7	60.9	125.0
	PCGWO	94.8	283.8	167.0	67.8	130.2
	BUJSO	106.8	288.7	166.1	64.8	141.3
	EPR-BUJSO	**66.2**	256.3	**127.0**	67.8	**92.8**

EPR-BUJSO achieves the lowest minimum, mean, and median makespan across all methods. For load imbalance, it leads on minimum, mean, and median, with a 23.6% lower mean than BUJSO (127.0 vs. 166.1). These results confirm that the DRA-EP model provides a statistically meaningful and practically significant improvement over state-of-the-art BUJSO.

5 Conclusion

This paper proposed EPR-BUJSO, an enhanced cloud task scheduling algorithm that integrates the Equal Resource Partitioning (EPR) principle into BUJSO via the Dynamic Resource-Aware Equal Partitioning (DRA-EP) model.

The DRA-EP model makes three formal contributions. Definition 1 establishes a normalized, dimensionally consistent Resource-Demand Density ρ_i that resolves the unit inconsistency present in prior work. Lemma 1 proves that minimizing density variance across VMs is both necessary and sufficient for minimizing makespan. Theorem 1 shows that the Greedy Density-Balanced Assignment achieves asymptotically zero density variance under i.i.d. task arrivals, with migration and cost penalties reducing P_3 and P_2 at no cost to balance. Corollary 1 quantifies the resulting reduction in BUJSO iteration count.

Experiments on 100-task, 10–50 VM configurations demonstrated that EPR-BUJSO achieves the lowest makespan at every VM count (3–21% over BUJSO), the lowest load

imbalance at four of five VM counts (up to 38.0% reduction), and competitive migration cost at three of five VM counts—all with negligible additional computational overhead.

Future work will extend EPR to heterogeneous cloud-edge environments, integrate ML-based workload prediction to refine GDBA density estimates dynamically, explore Pareto multi-objective formulations optimizing makespan and energy jointly, and evaluate on real-world CloudSim traces.

References

[1] Y. Pachipala, D. B. Dasari, V. V. R. M. Rao, P. Bethapudi, and T. Srinivasarao, "Workload prioritization and optimal task scheduling in cloud: introduction to hybrid optimization algorithm," *Wireless Networks*, 2024. https://doi.org/10.1007/s11276-024-03793-3

[2] D. Alsadie, "A metaheuristic framework for dynamic virtual machine allocation with optimized task scheduling in cloud data centers," *IEEE Access*, vol. 9, pp. 74218–74233, 2021.

[3] M. S. Ajmal, Z. Iqbal, F. Z. Khan, M. Ahmad, I. Ahmad, and B. B. Gupta, "Hybrid ant genetic algorithm for efficient task scheduling in cloud data centers," *Computers and Electrical Engineering*, vol. 95, p. 107419, 2021.

[4] S. E. Shukri, R. Al-Sayyed, A. Hudaib, and S. Mirjalili, "Enhanced multi-verse optimizer for task scheduling in cloud computing environments," *Expert Systems with Applications*, vol. 168, p. 114230, 2021.

[5] S. Velliangiri, P. Karthikeyan, V. A. Xavier, and D. Baswaraj, "Hybrid electro search with genetic algorithm for task scheduling in cloud computing," *Ain Shams Engineering Journal*, vol. 12, pp. 631–639, 2021.

[6] M. Sharma and R. Garg, "An artificial neural network based approach for energy efficient task scheduling in cloud data centers," *Sustainable Computing: Informatics and Systems*, vol. 26, p. 100373, 2020.

[7] H. Mahmoud, M. Thabet, M. H. Khafagy, and F. A. Omara, "Multiobjective task scheduling in cloud environment using decision tree algorithm," *IEEE Access*, vol. 10, p. 36140, 2022.

[8] S. Mangalampalli, G. R. Karri, and U. Kose, "Multi objective trust aware task scheduling algorithm in cloud computing using Whale optimization," *Journal of King Saud University – Computer and Information Sciences*, vol. 35, pp. 791–809, 2023.

[9] P. Tamilarasu and G. Singaravel, "Quality of service aware improved coati optimization algorithm for efficient task scheduling in cloud computing environment," *Journal of Engineering Research*, 2023.

[10] M. Maashi et al., "Elevating survivability in next-gen IoT-Fog-Cloud networks: scheduling optimization with the metaheuristic mountain gazelle algorithm," *IEEE Transactions on Consumer Electronics*, vol. 70, pp. 3802–3809, 2024.

[11] X. Chen, L. Cheng, C. Liu, Q. Liu, J. Liu, Y. Mao, and J. Murphy, "A WOA-based optimization approach for task scheduling in cloud computing systems," *IEEE Systems Journal*, vol. 14, pp. 3117–3128, 2020.

[12] R. L. Graham, "Bounds on multiprocessing timing anomalies," *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, 1969.